

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_12

12. Flyweight Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Flyweight pattern.

GoF Definition

It uses sharing to support large numbers of fine-grained objects efficiently.

Concept

This pattern may look simple but if you do not identify the core concepts, the implementations may appear to be complex. So, I'll start with a very basic and detailed explanation before you implement this pattern. Let's start.

Sometimes you need to handle lots of objects that are very similar (but not the same). But the constraint is that you cannot create all of them to lessen resource and memory usage. The Flyweight pattern is made to handle these scenarios.

So how do you do this? Ok, to understand this, let's quickly revisit the fundamentals of object-oriented programming. A class is a template or blueprint, and an object is an instance of it. An object can have states and behaviors. For example, if you are familiar with the game of football (or soccer, as it's known in the United States), you can say that Ronaldo or Beckham are objects from the `Footballer` class. You may notice that they have states like "playing state" or "non-playing state." In the playing state, they can show different skills (or behaviors)—they can run, they can kick, they can pass the ball, and so forth. To begin with object-oriented programming, you can ask the following questions:

- What are the possible states of my objects?

- What are the different functions (behaviors) they can perform in those states?

Once you get the answers to these questions, you are ready to proceed. Now come back to the Flyweight pattern. Here your job is to identify the following:

- What are the states of my objects?
- Which parts of these states can be changed?

Once you identify the answers, you break the states into two parts called **intrinsic** (which does not vary) and **extrinsic** (which can vary). If you make objects with intrinsic states, you can share them. For the extrinsic part, the user or client needs to pass in the information. So, whenever you need to have an object, you can get the object with the intrinsic state(s) and then you can configure the object on the fly by passing the extrinsic state(s). This technique reduces unnecessary object creations and memory usage.

Now verify your understanding in the following paragraph, which is extremely important. Let's see what GoF said about flyweights:

A flyweight is a shared object that can be used in multiple contexts simultaneously. The flyweight acts as an independent object in each context—it's indistinguishable from an instance of the object that's not shared. Flyweights cannot make assumptions about the context in which they operate. The key concept here is the distinction between intrinsic and extrinsic states. The intrinsic state is stored in the flyweight; it consists of information that's independent of the flyweight's context, thereby making it sharable. The extrinsic state depends on and varies with the flyweight's context and therefore can't be shared. Client objects are responsible for passing extrinsic state to the flyweight when it needs it.

Real-Life Example

Suppose you have a pen. You can use different ink refills to write with different colors. So, the pen without the refill can be considered the flyweight with intrinsic data, and the refills can be considered the extrinsic data in this example. Now consider the case when you have many similar pens with different ink fills. For simplicity, assume that you have some pens with red refills and some pens with green refills and you want to distribute these pens among the kids. Later you identify that the kids like red pens more than green pens. So, you replace the green refills with red

refills to make more red pens and distribute them among the kids to make them happy.

Consider another example. Suppose a company needs to print business cards for its employees. In this case, what is the starting point? The print department can create a common template where the company logo, address, and other details are already printed (intrinsic) and later the company adds the particular employee details (extrinsic) on the cards.

Computer World Example

Consider a computer game where you see a large number of participants whose core structures are the same, but their appearances vary (for example, they may have different states, colors, weapons, and so on). Therefore, when the game runs, if you want to store all of these objects with all of these variations/states, the memory requirement will be huge. So, instead of storing all these objects, you can design the application in such a way that you will create one of these instances with the states that don't vary among objects and your client can maintain remaining variations/states. When you successfully implement the concept, you can say that you have followed the flyweight pattern in your application. Here are few more examples:

- In Java, you may notice the use of this pattern when you use wrapper classes like `java.lang.Integer`, `java.lang.Short`, `java.lang.Byte`, and `java.lang.Character` where the static method `valueOf()` replicates a factory method. (It is worth remembering that some wrapper classes such as `java.lang.Double` do not use the same concept.) This method checks whether an object is present in the cache. If it is found in the cache, it is returned.
- A String pool in JVM is another example of a flyweight. When you call the `intern()` method, it can return a cached object, too.
- Similarly, the string interning in a .NET application follows this pattern.

Implementation

The following example shows the usage of three different types of vehicles called `Car`, `Bus`, and `FutureVehicle` (let's assume we'll see them in 2050). In this application, you assume that a client may want to use a large number of objects from these classes with different colors that they like. You also assume that the basic structure of a car (or bus, etc.) does not vary.

When a client requests a particular vehicle, the application will not create an object from scratch if it previously created an instance of that type of vehicle; instead, it'll prepare the existing one (without `color`) to serve the needs of the client. Just before delivering the product, it'll paint the vehicle with the `color` that the client prefers. Now look into the implementation strategies, which are as follows.

First, you create an interface for flyweights. This interface is made to provide common method(s) that accept extrinsic state(s) of flyweights. In your example, `color` is supplied by clients, so it is treated as an extrinsic state. This is why you see the following code segment:

```
interface Vehicle {  
    // Color comes from client. It is extrinsic.  
    void aboutMe(String color);  
}
```

Most often you will see a factory that will supply the flyweights to the client. This factory caches flyweights and provides methods to get them. In a shared flyweight object, you add intrinsic state(s) and implement the method(s) if necessary. You can have unshared flyweights, too. In these cases, you can ignore the extrinsic states that are passed by a client.

In the upcoming example, `VehicleFactory` is the factory that supplies the flyweights with intrinsic states. Notice that a `Map` object is used to store the `key/value` pairs to store vehicles with a specific type. Initially, there are no objects inside the factory, but once it starts receiving requests for vehicles, it will create the vehicles and cache them for future use. Notice that “One car is ready,” “One bus is ready,” and “One future vehicle(`Vehicle 2050`) is ready” are supplied by the factory inside the flyweight objects during the object creation phase. They are the intrinsic states of the vehicles and they don't vary across the products.

In addition, there is a method called `totalObjectsCreated()` that is used to track the number of truly unique objects in the application. The following code segment shows this factory class:

```
class VehicleFactory {  
    Map<String, Vehicle> vehicles = new HashMap<String,  
                                                Vehicle>();  
  
    /**  
     * To count different types of vehicles
```

```

        * in a given moment.
        */
public int totalObjectsCreated() {
    return vehicles.size();
}
public synchronized Vehicle
    getVehicleFromFactory(String vehicle)
                                throws Exception {
    Vehicle vehicleType = vehicles.get(vehicle);
    if (vehicleType != null) {
        // Using the existing type
        System.out.println("\n\t Using an existing vehicle now."
    } else {
        switch (vehicle) {
            case "car":
                System.out.println("Making a car for the first time
                vehicleType = new Car("One car is ready");
                vehicles.put("car", vehicleType);
                break;
            case "bus":
                System.out.println("Making a bus for the first time
                vehicleType = new Bus("One bus is ready");
                vehicles.put("bus", vehicleType);
                break;
            case "future":
                System.out.println("Making a future
                                vehicle for the first time.");
                vehicleType = new FutureVehicle("One
                                future vehicle(Vehicle 2050) is ready");
                vehicles.put("future", vehicleType);
                break;
            default:
                throw new Exception("Unknown vehicle type.");
        }
    }
    return vehicleType;
}
}

```

Let's see a concrete flyweight class now. Here is one of these classes (the others are similar). The associated comments will help you understand it better. Notice how the method `aboutMe()` is used to show the intrinsic and extrinsic states of the vehicle.

Note The comments are here for your easy reference. In the demonstration, I removed them to avoid repetition. But when you download the

code from the Apress website, you get them again.

```
// A shared flyweight implementation
class Car implements Vehicle {
    /**
     * This is an intrinsic state:
     * 1.It is not supplied by client.
     * 2.It is independent of the flyweight's context.
     * 3.This can be shared across.
     * 4.These data are often immutable.
     */
    private String description;
    /**
     * The VehicleFactory will supply this
     * inside the flyweight object.
     */
    public Car(String description) {
        this.description = description;
    }
    // Client supplies the color
    @Override
    public void aboutMe(String color) {
        System.out.print(description + " with " + color
            + " color.");
    }
}
```

From this code segment, you can see that the `description` is supplied during the object creation process (the Flyweight factory does this), but the `color` will be supplied by the clients. In this example, the colors are drawn at random using a method called `getRandomColor()`. This method has the following definition:

```
/**
 * Get a random color. Here I consider
 * two colors only-red and green.
 */
static String getRandomColor() {
    Random r = new Random();
    int random = r.nextInt();
    if (random % 2 == 0) {
        return "red";
    } else {
        return "green";
    }
}
```

So, inside the `main()` method, note the code segments like the following:

```
vehicle.aboutMe(getRandomColor());
```

Lastly, the `FutureVehicle` is considered as an unshared flyweight in this example. So, in this class, the `aboutMe(...)` method ignores the `String` parameter. As a result, it always produces vehicles with a blue color and ignores the client's preferences. Here is the code segment with comments that reflects the same:

```
// Client cannot choose color for FutureVehicle.
// It is an unshared flyweight.
// So, we ignore the client's input.
@Override
public void aboutMe(String color) {
    System.out.print(description + " with " + "blue
        (default) color.");
}
```

Class Diagram

Figure 12-1 shows the class diagram.

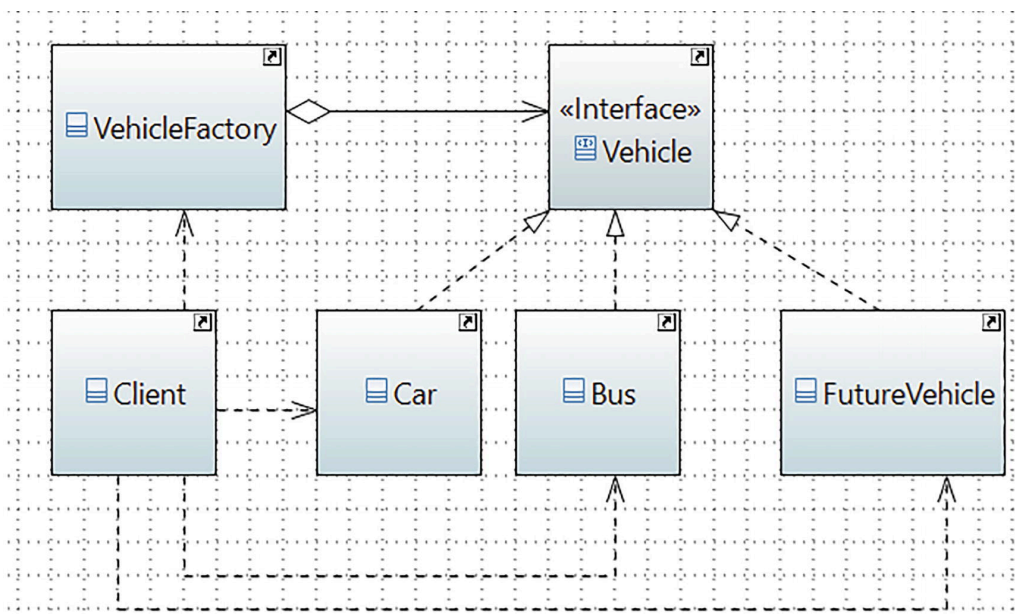


Figure 12-1 Class diagram

Package Explorer View

Figure 12-2 shows the high-level structure of the program. Since it's a big diagram, let's move to the next page.

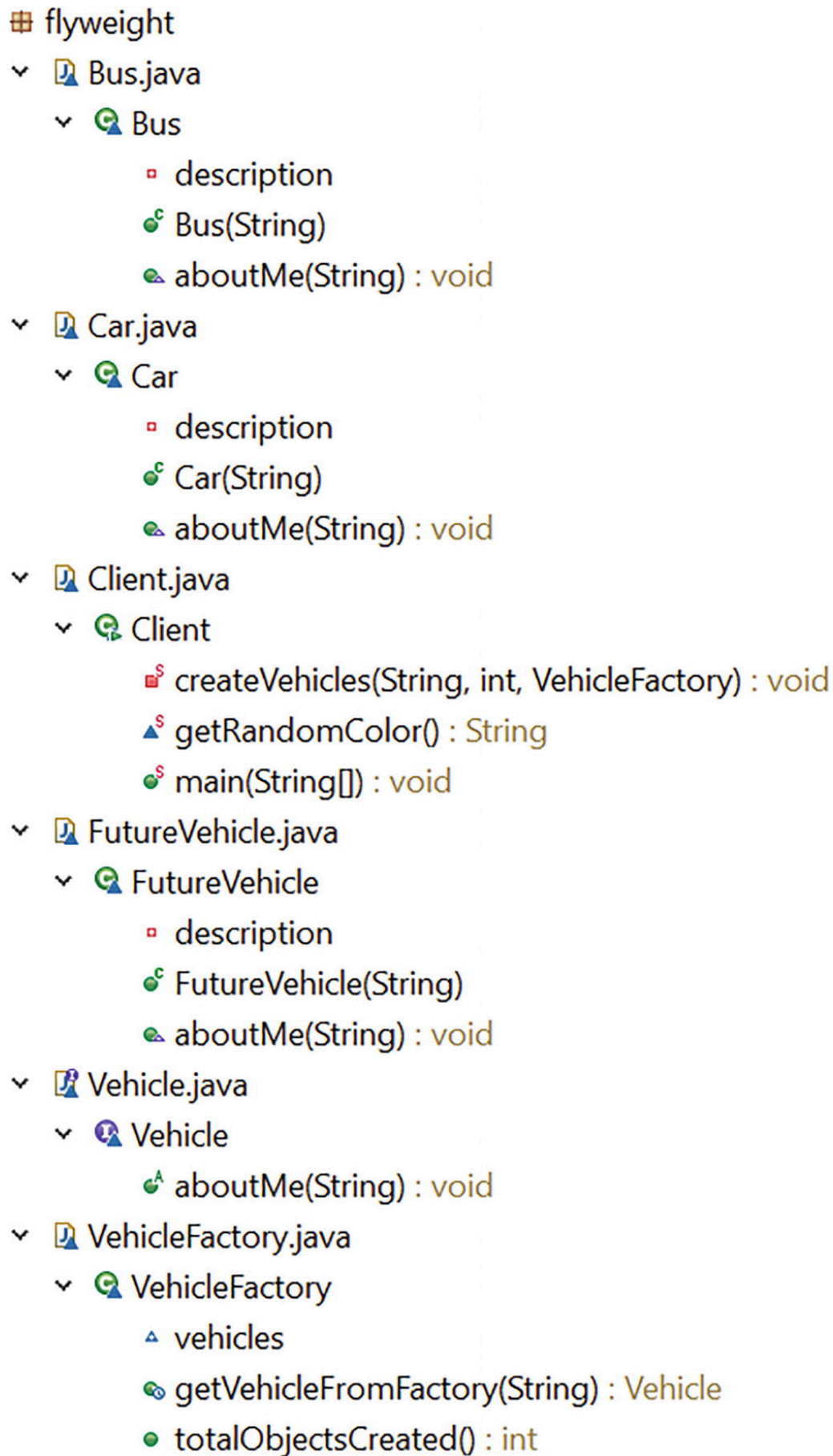


Figure 12-2 Package Explorer view

Demonstration

All parts of the program are stored inside the package `jdp3e.flyweight`. Here's the complete implementation:

```
// Vehicle.java
interface Vehicle {
    // Color comes from client. It is extrinsic.
    void aboutMe(String color);
}

// Car.java
// A shared flyweight implementation
class Car implements Vehicle {
    private String description;
    public Car(String description) {
        this.description = description;
    }
    @Override
    public void aboutMe(String color) {
        System.out.print(description + " with " + color
            + " color.");
    }
}

// Bus.java
// A shared flyweight implementation
class Bus implements Vehicle {
    private String description;
    public Bus(String description) {
        this.description = description;
    }
    @Override
    public void aboutMe(String color) {
        System.out.print(description + " with " + color
            + " color.");
    }
}

// FutureVehicle.java
// An unshared flyweight implementation
class FutureVehicle implements Vehicle {
    private String description;
    public FutureVehicle(String description) {
        this.description = description;
    }
    // Client cannot choose color for FutureVehicle.
    // It is an unshared flyweight.
    // So, we ignore the client's input.
    @Override
    public void aboutMe(String color) {
        System.out.print(description + " with " + "a
```

```

        blue (default) color.");
    }
}

// VehicleFactory.java
import java.util.HashMap;
import java.util.Map;
class VehicleFactory {
    Map<String, Vehicle> vehicles = new HashMap<String,
                                                Vehicle>();

    /**
     * To count different types of vehicles
     * in a given moment.
     */
    public int totalObjectsCreated() {
        return vehicles.size();
    }

    public synchronized Vehicle
    getVehicleFromFactory(String vehicle)
        throws Exception {
        Vehicle vehicleType = vehicles.get(vehicle);
        if (vehicleType != null) {
            // Using the existing type
            System.out.println("\n\t Using an existing vehicle now."
        } else {
            switch (vehicle) {
                case "car":
                    System.out.println("Making a car for the first time
                    vehicleType = new Car("One car is ready");
                    vehicles.put("car", vehicleType);
                    break;
                case "bus":
                    System.out.println("Making a bus for the first time
                    vehicleType = new Bus("One bus is ready");
                    vehicles.put("bus", vehicleType);
                    break;
                case "future":
                    System.out.println("Making a future
                    vehicle for the first time.");
                    vehicleType = new FutureVehicle("One
                    future vehicle(Vehicle 2050) is ready");
                    vehicles.put("future", vehicleType);
                    break;
                default:
                    throw new Exception("Unknown vehicle type.");
            }
        }
        return vehicleType;
    }
}

```

```

    }

}

// Client.java
import java.util.Random;

class Client {
    public static void main(String[] args) throws
        Exception {
        System.out.println("***Flyweight Pattern Demo.***\n");
        VehicleFactory vehicleFactory = new
            VehicleFactory();

        // Making 3 cars
        createVehicles("car", 3, vehicleFactory);
        // Making 5 Buses
        createVehicles("bus", 5, vehicleFactory);
        // Making 2 future vehicles
        createVehicles("future", 2, vehicleFactory);
    }

    /**
     * We are trying to get the 5 vehicles.
     * Note that: we need not create additional
     * vehicles if we have already created one of these
     * categories.
     */
    private static void createVehicles(String
        vehicleType, int count, VehicleFactory factory)
        throws Exception {

        int distinctVehicles;
        for (int i = 0; i < count; i++) {
            Vehicle vehicle =
                factory.getVehicleFromFactory(vehicleType);
            vehicle.aboutMe(getRandomColor());
        }
        distinctVehicles =
            factory.totalObjectsCreated();
        System.out.println("\n\tDistinct vehicles in
            this application: " + distinctVehicles);
        System.out.println("-----");
    }

    /**
     * Get a random color. Here I consider
     * two colors only-red and green.
     */
    static String getRandomColor() {
        Random r = new Random();
        int random = r.nextInt();
        if (random % 2 == 0) {
            return "red";
        }
    }
}

```

```

        } else {
            return "green";
        }
    }
}

```

Output

Here's a possible output (because the `color` is generated at random). This is from the first run on my machine. I marked a few lines bold to show the unique number of vehicles at a given time.

```

***Flyweight Pattern Demo.***
Making a car for the first time.
One car is ready with green color.
    Using an existing vehicle now.
One car is ready with green color.
    Using an existing vehicle now.
One car is ready with red color.
    Distinct vehicles in this application: 1
-----
Making a bus for the first time.
One bus is ready with green color.
    Using an existing vehicle now.
One bus is ready with red color.
    Using an existing vehicle now.
One bus is ready with green color.
    Using an existing vehicle now.
One bus is ready with green color.
    Using an existing vehicle now.
One bus is ready with red color.
    Distinct vehicles in this application: 2
-----
Making a future vehicle for the first time.
One future vehicle(Vehicle 2050) is ready with a blue (default) color.
    Using an existing vehicle now.
One future vehicle(Vehicle 2050) is ready with a blue (default) color.
    Distinct vehicles in this application: 3
-----

```

Here's another probable output (because the `color` is generated at random). This is from the second run on my machine.

```

***Flyweight Pattern Demo.***
Making a car for the first time.
One car is ready with green color.
    Using an existing vehicle now.
One car is ready with green color.
    Using an existing vehicle now.
One car is ready with green color.
    Distinct vehicles in this application: 1
-----
Making a bus for the first time.
One bus is ready with green color.
    Using an existing vehicle now.
One bus is ready with green color.
    Using an existing vehicle now.
One bus is ready with green color.
    Using an existing vehicle now.
One bus is ready with green color.
    Using an existing vehicle now.
One bus is ready with green color.
    Using an existing vehicle now.
One bus is ready with green color.
    Distinct vehicles in this application: 2
-----
Making a future vehicle for the first time.
One future vehicle(Vehicle 2050) is ready with a blue (default) color.
    Using an existing vehicle now.
One future vehicle(Vehicle 2050) is ready with a blue (default) color.
    Distinct vehicles in this application: 3
-----

```

Note Remember that the outputs vary because the colors are picked at random in this example.

Analysis

Notice that the application creates an object if and only if the object is not available at that moment. Once it creates an object, it caches the object for future reuse.

Q&A Session

12.1 Can you highlight the key differences between a Singleton pattern and a Flyweight pattern?

Singleton helps you to maintain at most one object that is required in the system. In other words, once the required object is created, you cannot

create more of it. You need to reuse the existing object.

Normally, a Flyweight pattern concerns heavy but similar objects (where all states are not exactly the same) because they may occupy big blocks of memory. So, you try to create a smaller set of template objects that can be configured on the fly to make these heavy objects. These smaller and configurable objects are called flyweights. You can reuse them in your application when you deal with many large objects. This approach helps you reduce the consumption of big chunks of memory. Basically, flyweights make *one look like many*. This is why the GoF tells us the following: “A flyweight is a shared object that can be used in multiple contexts simultaneously. The flyweight acts as an independent object in each context—it’s indistinguishable from an instance of the object that’s not shared.”

Figure 12-3 shows you how to visualize the core concepts of the Flyweight pattern before using flyweights.

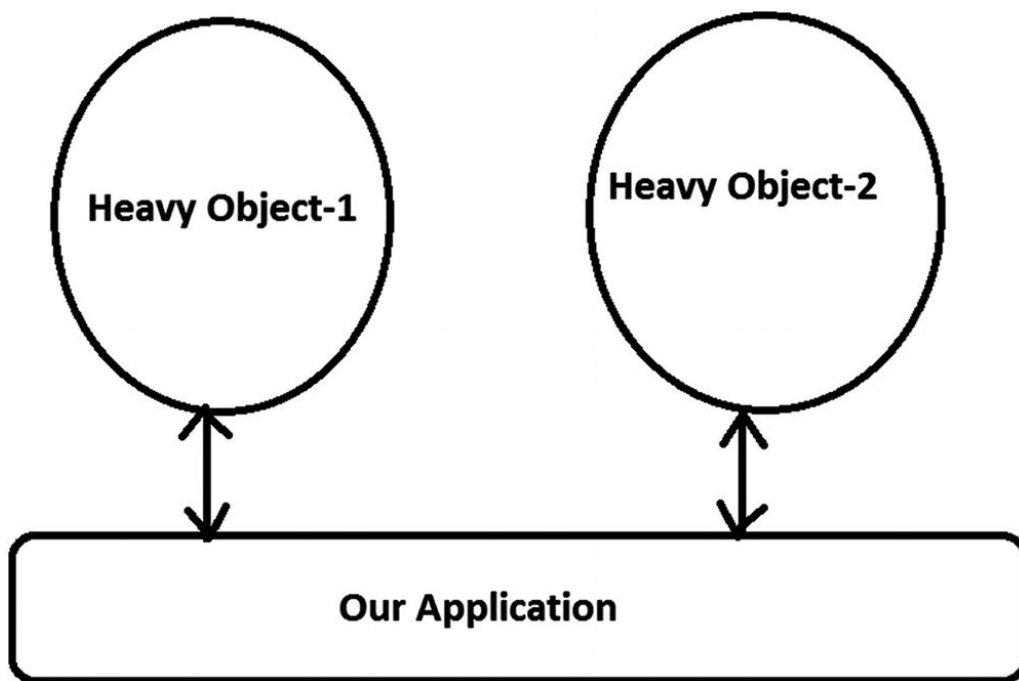


Figure 12-3 Before using flyweights

Figure 12-4 shows the design after using flyweights.

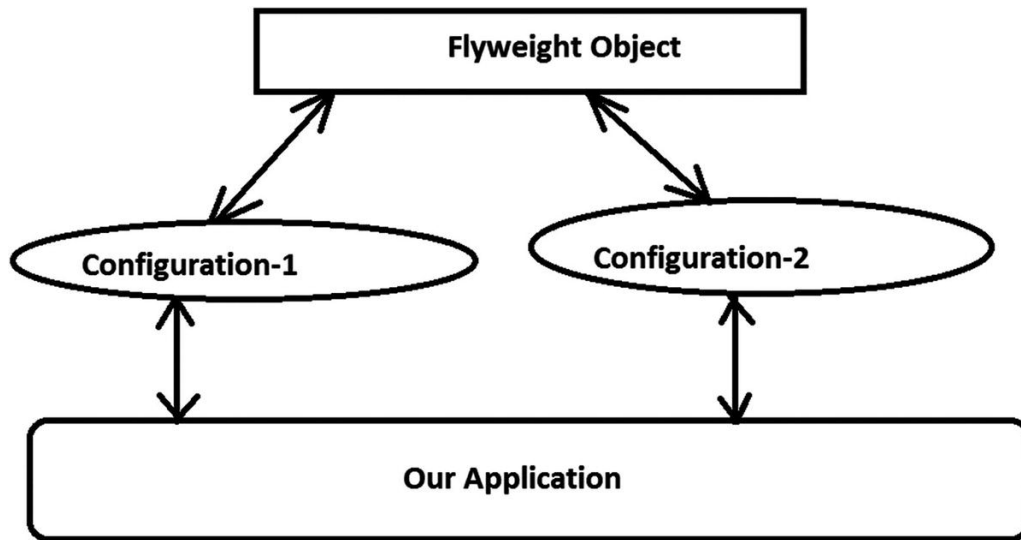


Figure 12-4 After using flyweights

So, from Figure [12-4](#), you can see that the Heavy-Object1 can be created when you apply Configuration-1 to the Flyweight Object. Similarly, Heavy-Object2 can be created when you apply Configuration-2 to the Flyweight Object. You can see that instance-specific contents (like color in the previous demonstration) can be passed to the flyweights to make these heavy objects. So, note in this example that the flyweight object acts like a common template that can be configured as per the need.

12.2 Is there any impact because of multithreading?

If you create objects with a new operator, in a multithreaded environment you may end up creating multiple unwanted objects. This kind of challenge is similar to the Singleton pattern and the remedy is also similar.

12.3 What are the advantages of using the Flyweight design pattern?

Here are some advantages:

- You can reduce the memory consumption of heavy objects that can be controlled identically.
- You can reduce the total number of unique objects in the system.
- You can maintain centralized states of many “virtual” objects.

12.4 What are the challenges associated with using the Flyweight design pattern?

Here are some challenges:

- In this pattern, you need to spend some time configuring these flyweights. This configuration time can impact the overall performance of the application.
- To create flyweights, you extract a common template class from existing objects. This additional layer of programming can be tricky and sometimes hard to debug and maintain.

12.5 Can you have a non-shareable flyweight interface?

Yes, a flyweight interface does not enforce that it needs to be shareable always. So, in some cases, you may have non-shareable flyweights only. In our example, the `FutureVehicle` class is made to demonstrate this case. You can see that its instances are always made with a blue color. For this vehicle, it doesn't matter whatever color (red or green or any other color) a client supplies to it as an extrinsic state.

12.6 Since the intrinsic data of flyweights is the same, you try to share them. Is this correct?

Yes. Notice that the strings “One car is ready,” “One bus is ready,” or “One future vehicle(`Vehicle 2050`) is ready” are supplied by the factory inside the flyweights during the flyweight (with the intrinsic state) object creation phase.

12.7 How do clients handle the extrinsic data of these flyweights?

They need to pass that information (the states) to the flyweights.

12.8 This means that extrinsic data is not shareable. Is this correct?

Yes. It's very important to understand this before you implement this pattern.

12.9. What is the role of `VehicleFactory` in this implementation?

It caches flyweights and provides a method to get them. In this example, there are multiple objects with an intrinsic state that can be shared. I believe that storing them in a central place is a good idea.

12.10 Can I implement the factory class as a singleton?

Yes, you can. In fact, you may see this in many applications. In addition, you'll see some of those applications instantiate all possible objects at the

beginning. For example, you may try to initialize all different types of vehicles inside the `VehicleFactory`'s constructor. When you do that, you always start with three distinct vehicle objects at the beginning. But the problem is, if you do not need bus, car, or `Vehicle2050`, you unnecessarily waste memory for the object(s).

On the contrary, in the shown demonstration, if any object is not available, the factory class creates it and caches it for future use. So, my vote is for this kind of demonstration unless you modify such an implementation keeping this potential drawback in mind.

In short, whenever you use this pattern, you create an object, fill in all the required state information, and give it to your client. Each time a client requests an object, your application should check whether it can reuse an existing object (with required states filled) or not. Thus you can reduce unnecessary object creations and save memory consumption.

12.11 You said that `java.lang.Integer` is an example of flyweight. Can you please elaborate?

Let me show you a partial snapshot from the Eclipse IDE of the `valueOf(...)` method inside the `Integer` class (see Figure 12-5).

```
* This method will always cache values in the range -128 to 127,
* inclusive, and may cache other values outside of this range.
*
* @param i an {@code int} value.
* @return an {@code Integer} instance representing {@code i}.
* @since 1.5
*/
@IntrinsicCandidate
public static Integer valueOf(int i) {
    if (i >= IntegerCache.Low && i <= IntegerCache.high)
        return IntegerCache.cache[i + (-IntegerCache.Low)];
    return new Integer(i);
}
```

Figure 12-5 The definition of the `valueOf(...)` method inside the `Integer` class

Have you noticed the method description? Can you identify the words such as “`IntrinsicCandidate`” and “`cache`” here? What does the method do? It checks whether the integer argument belongs to a particular range. If it is in the range, you get a cached object.

(You may note that `public Integer(int value){...}` is deprecated since Java 9. It is reflected inside the snapshot too.)

You can also investigate the definition of the `intern()` method of `String` in the same way to get the following information:

“A pool of strings, initially empty, is maintained privately by the class `String`. When the `intern` method is invoked, if the pool already contains a string equal to this `String` object as determined by the `equals(Object)` method, then the string from the pool is returned. Otherwise, this `String` object is added to the pool, and a reference to this `String` object is returned.”

Previous chapter

[< 11. Facade Pattern](#)

Next chapter

[13. Composite Pattern >](#)